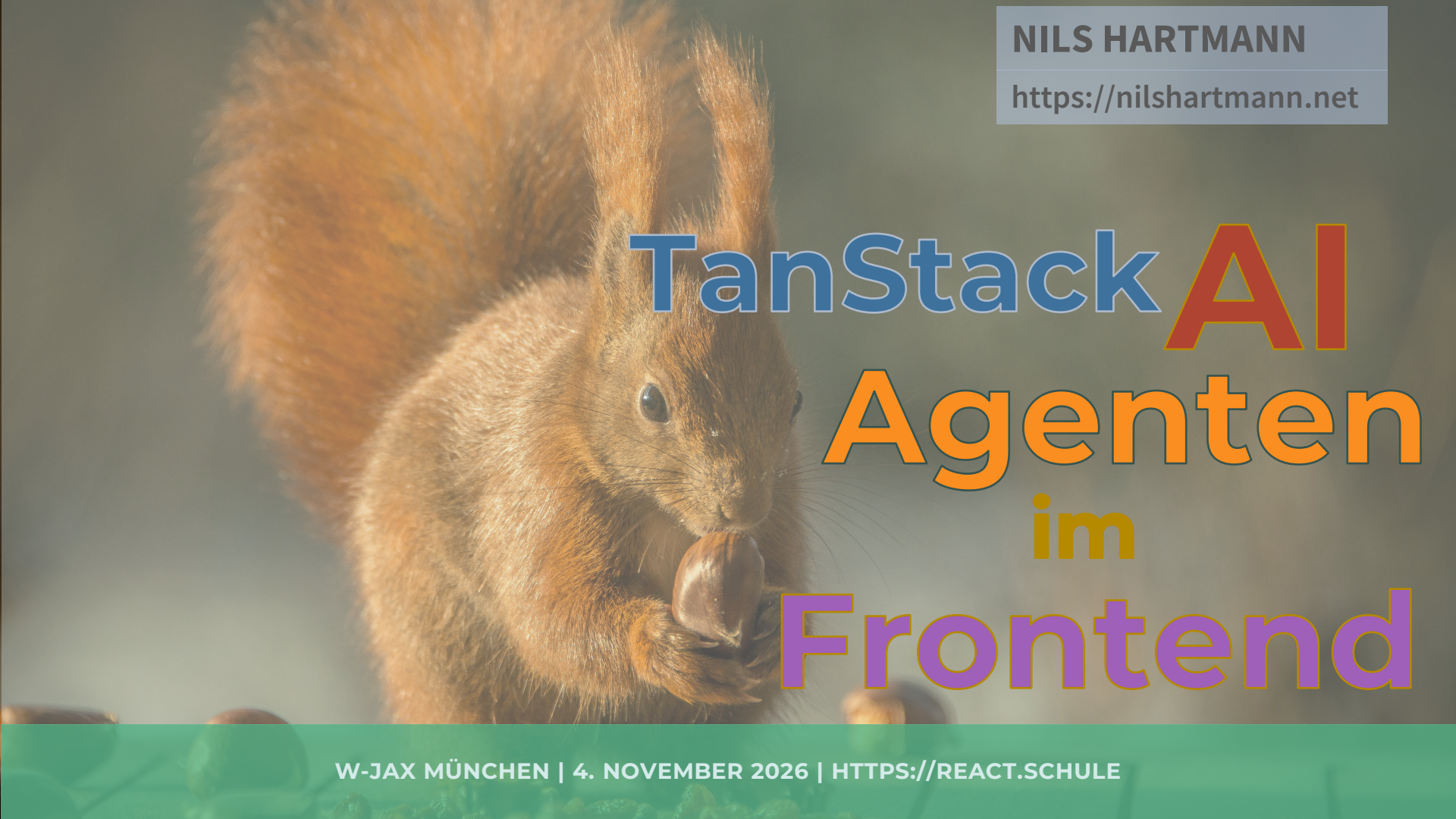


NILS HARTMANN
https://nilshartmann.net

TanStack AI Agenten im Frontend

W-JAX MÜNCHEN | 4. NOVEMBER 2026 | HTTPS://REACT.SCHULE

NILS HARTMANN

nils@nilshartmann.net

Freiberuflicher Software-Entwickler, –Architekt, Coach, Trainer

Java, Spring, GraphQL, React, TypeScript



https://react.schule/video-kurs-react



https://reactbuch.de

Content

Warning



Dieser Vortrag ist

mit der heißen Nadel gestrickt

(...und ich kann gar nicht stricken)

CONTENT WARNING

W-JAX 2026 - weiterer Talk



Christina Bergh Mi., 21. Okt., 17:35 (vor 13 Tagen)

an mich

Hei Nils,

die W-JAX steht vor der Tür und durch ungeplante Absagen haben wir jetzt leider einen offenen Slot am Donnerstagmorgen um 9:00 Uhr. Hättest Du vielleicht Zeit und Lust diesen Slot zu füllen mit einem Deiner Talks?

CONTENT WARNING



TANSTACK @tanstack.com · 10 T

TanStack AI 1.0 is officially stable! 🎉

Build chat, voice, image generation, agents, and harnesses into your apps, with your choice of models, UI framework, and infrastructure.

Your AI, your way.

CONTENT WARNING

nützt ja allens nix

Wat mutt, dat mutt

(plattdeutsch für: hilft ja nichts, da müssen wir jetzt durch)

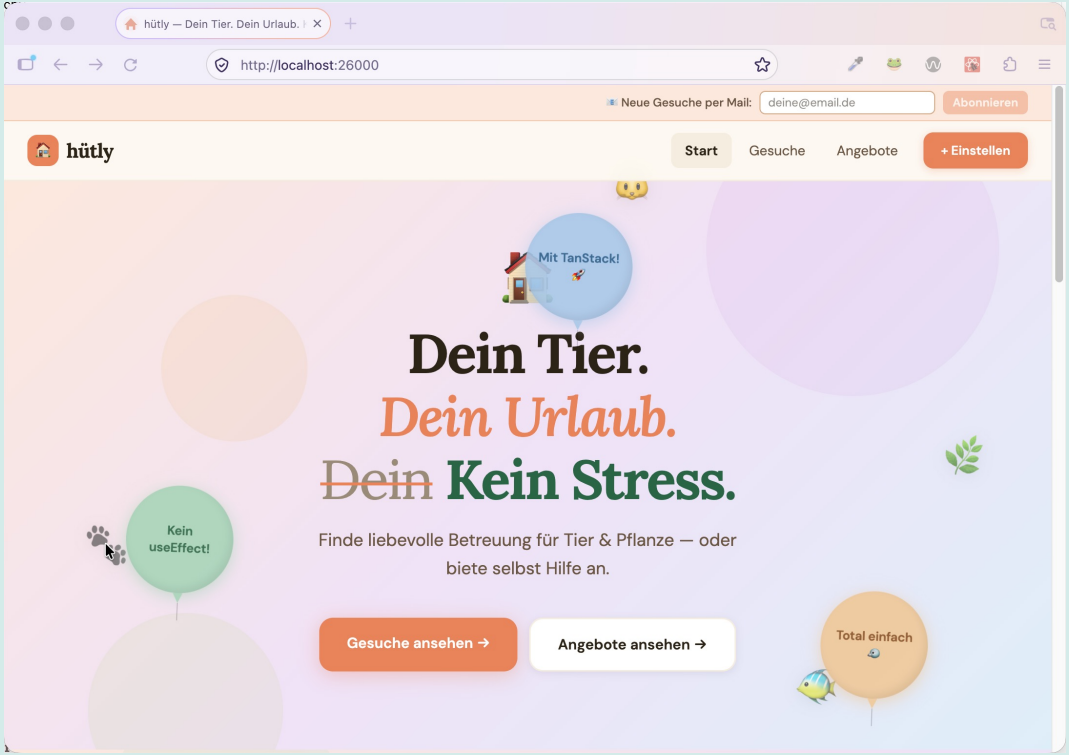
CONTENT WARNING

TanStack AI



Wer hat Haustiere?

EINE FRAGE...



HTTP://LOCALHOST:26000



BUSINESS ANALYSE...

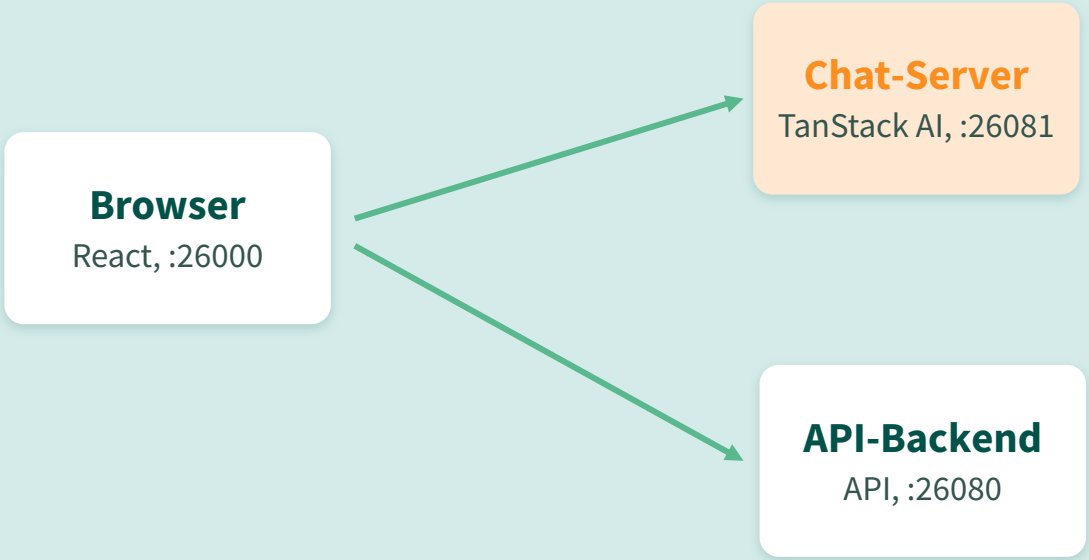


STRATEGIE: VIEL KI!

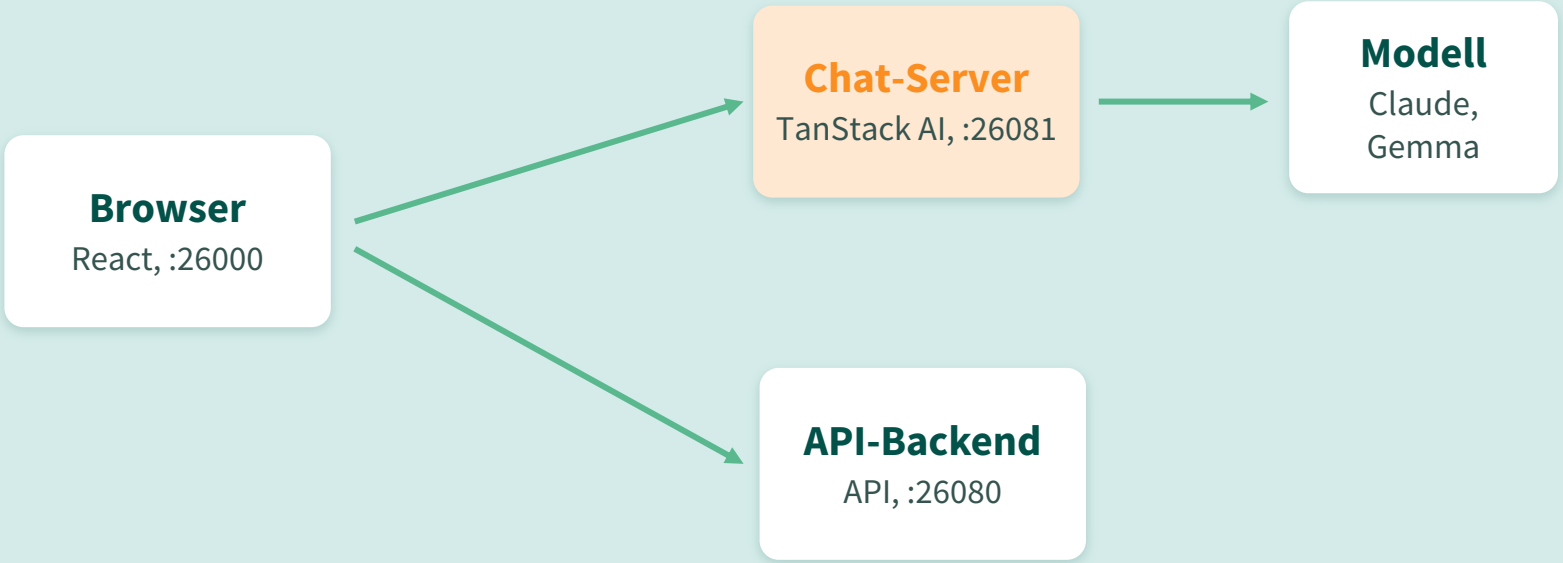
Die Architektur von hütly



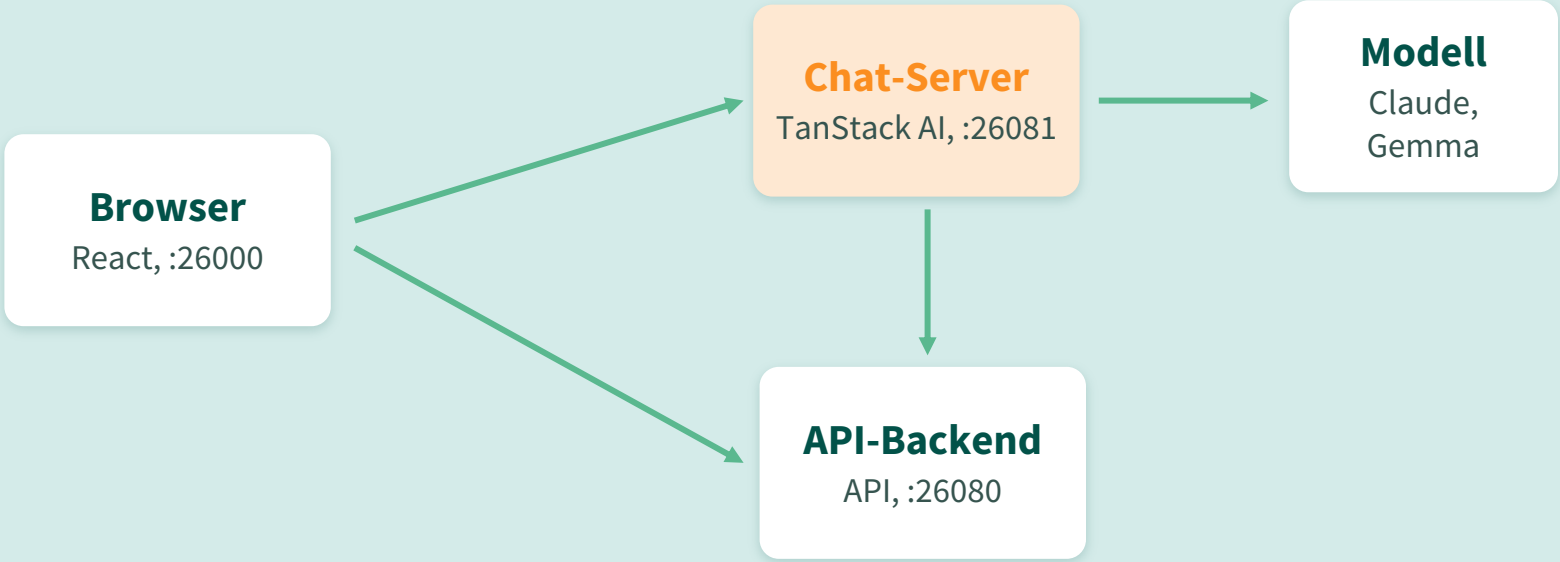
hütly bekommt einen Assistenten



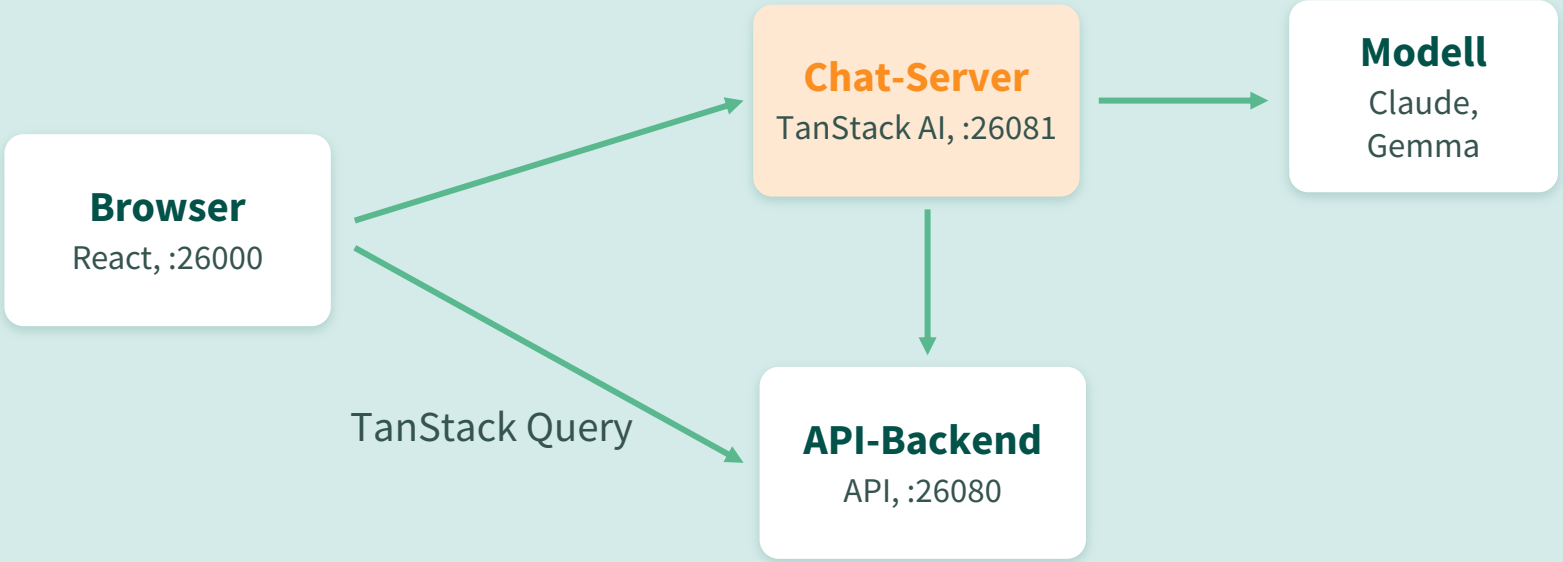
hütly bekommt einen Assistenten



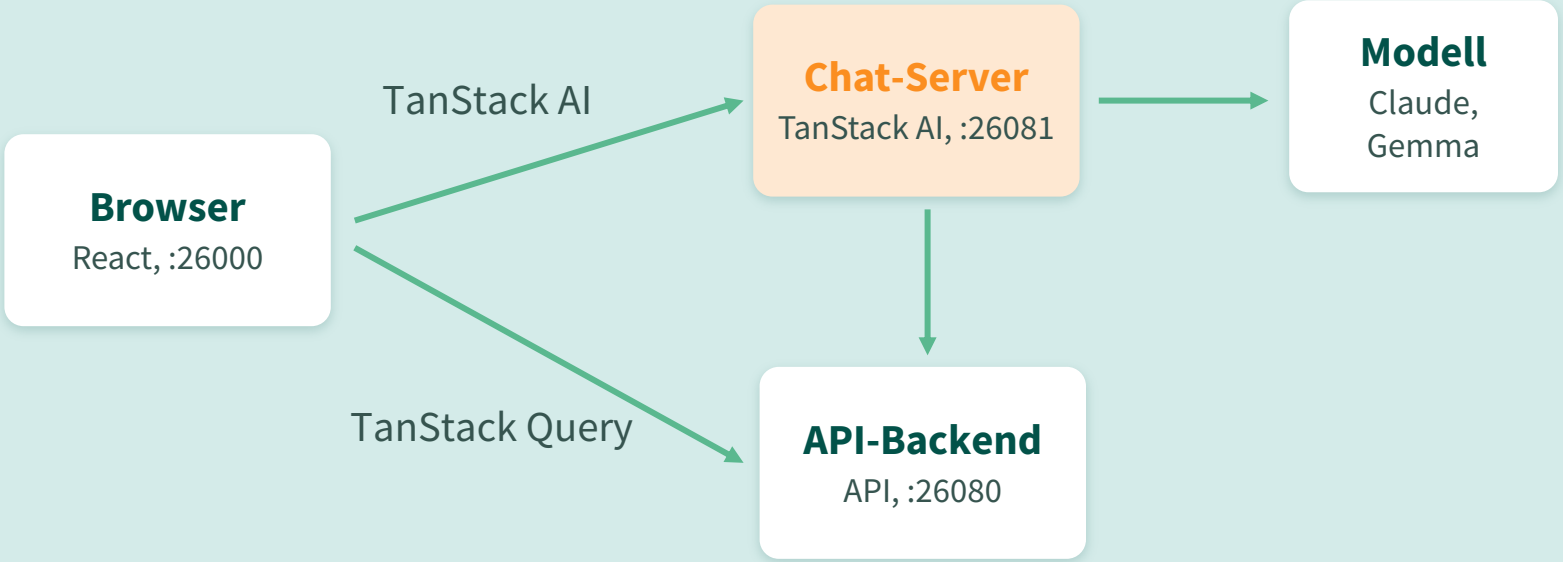
hütly bekommt einen Assistenten



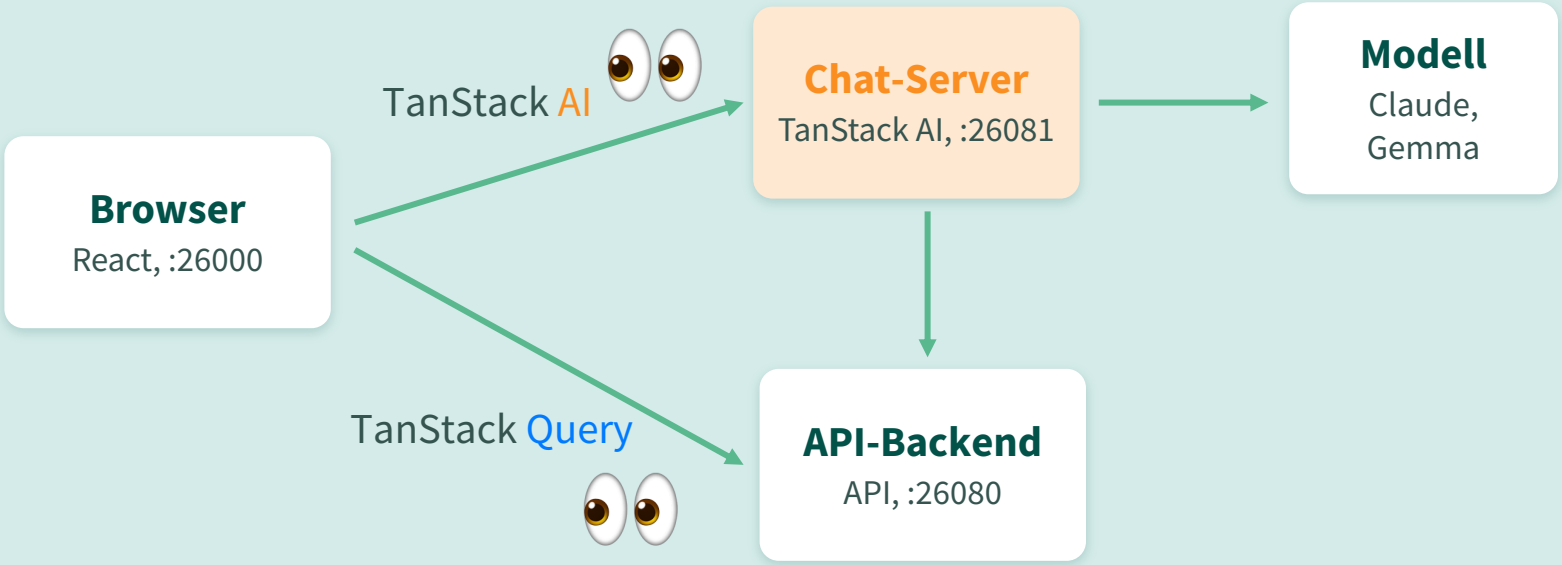
hütly bekommt einen Assistenten



hütly bekommt einen Assistenten



hütly bekommt einen Assistenten



Ein Agent ist keine gewöhnliche API



Antwortet stückweise

Der Text kommt Wort für Wort, nicht am Stück

Ein Agent ist keine gewöhnliche API



Antwortet stückweise

Der Text kommt Wort für Wort, nicht am Stück



Holt sich selbst Daten

Zwischendurch ruft er Tools auf, auf dem Server und im Browser

Ein Agent ist keine gewöhnliche API

✍️

Antwortet stückweise

Der Text kommt Wort für Wort, nicht am Stück

🔍

Holt sich selbst Daten

Zwischendurch ruft er Tools auf, auf dem Server und im Browser

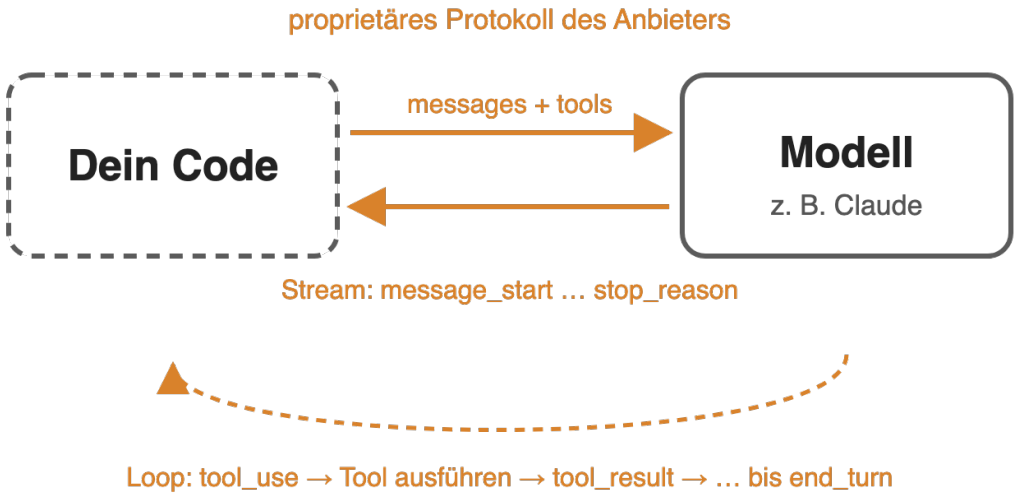
✋

Fragt manchmal nach

Vor einer Aktion hält er an und wartet auf einen Menschen

Die Strecke vom *Frontend* zum *Modell*

Wir sprechen mit dem Modell…



Wir sprechen mit dem Claude…

Hin: Anfrage

```
POST https://api.anthropic.com/v1/messages
x-api-key: …   anthropic-version: 2023-06-01

{ "model": "claude-sonnet-5-5", "stream": true,
  "system": "Du bist der Assistent von hütly …",
  "tools": [{ "name": "gesucheSuchen",
              "input_schema": { … } }],
  "messages": [
    { "role": "user", "content": "Katzen?" },
    { "role": "assistant", "content": [
      { "type": "tool_use", "id": "toolu_01", … } ] },
    { "role": "user", "content": [
      { "type": "tool_result",
        "tool_use_id": "toolu_01", … } ] } ] }
```

Zurück: Server-Sent Events

```
event: message_start
event: content_block_start
  { "type": "thinking" }
event: content_block_delta
  { "type": "signature_delta", … }
event: content_block_start
  { "type": "tool_use",
    "name": "gesucheSuchen" }
event: content_block_delta
  { "type": "input_json_delta",
    "partial_json": "{\"tierart\": \"Ka" }
event: message_delta
  { "stop_reason": "tool_use" }
event: message_stop
```

Die ganze Unterhaltung geht bei jeder Anfrage mit.

Demo: Anfrage ohne TanStack AI

- Claude direkt über die API von Anthropic aufrufen, mit curl
- Der System-Prompt ist nur ein Feld
- Die Antwort kommt als Server-Sent Events im Format von Anthropic



Und wenn wir das Modell austauschen?

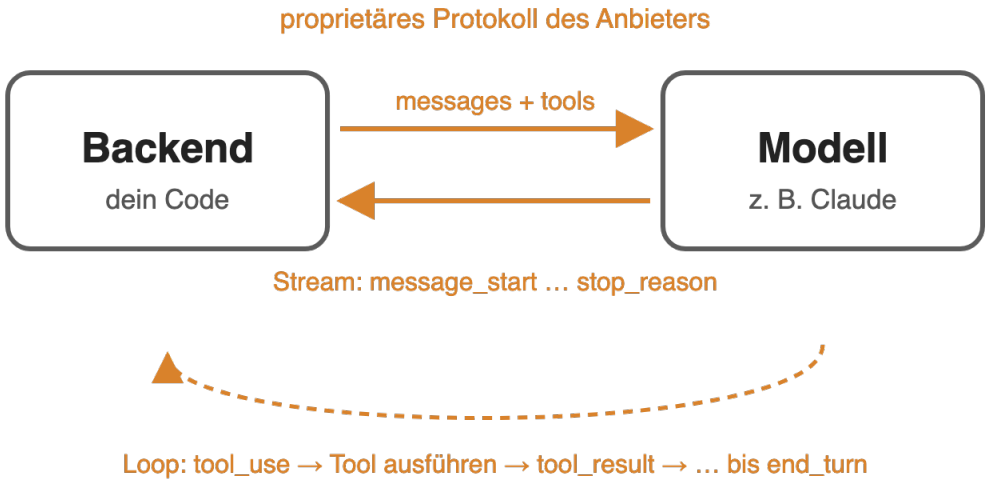
	Anthropic	OpenAI	Gemini	Ollama
Endpunkt	`/v1/messages`	`/v1/chat/completions`	`:streamGenerateContent`	`/api/chat`
Tool-Aufruf	`Block tool_use`	`message.tool_calls`	`Part functionCall`	`message.tool_calls`
„Ich will ein Tool“	`stop_reason: "tool_use"`	`finish_reason: "tool_calls"`	`functionCall im Part`	`tool_calls + done`
Tool-Ergebnis zurück	`tool_result, role user`	`role "tool", tool_call_id`	`Part functionResponse`	`role: "tool"`
Streaming	`SSE, benannte Events`	`SSE, data: … [DONE]`	`SSE mit alt=sse`	`NDJSON, kein SSE`

Was dein Code alles können müsste 🙀

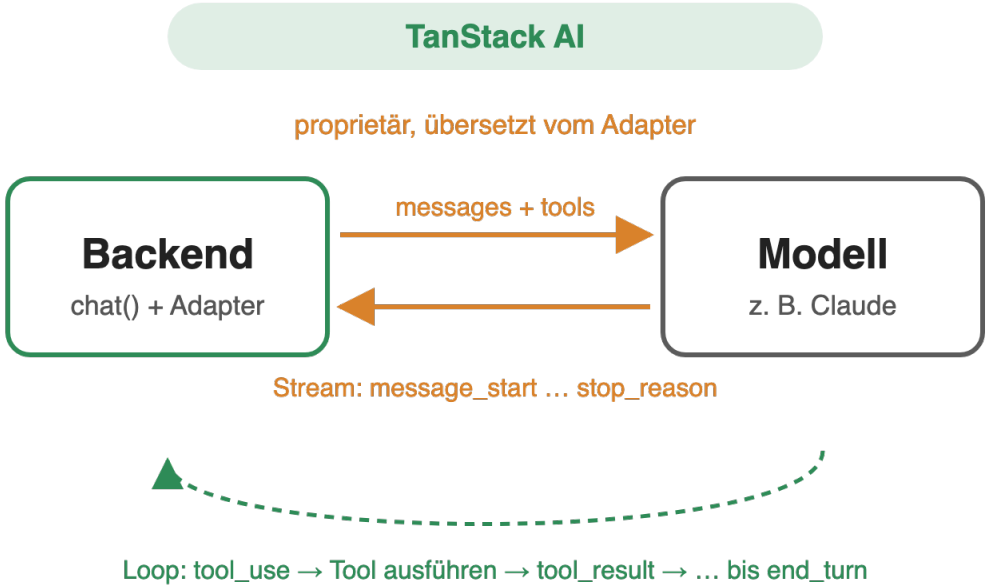
Dein Code

- [ ] HTTP-Client mit Schlüssel und Headern des Anbieters
- [ ] Stream lesen: SSE oder NDJSON zerlegen
- [ ] Deltas zu Text, Denken und Tool-Aufrufen zusammensetzen
- [ ] Tool-Argumente aus JSON-Schnipseln parsen und prüfen
- [ ] Den Loop drehen: Tool ausführen, Ergebnis zurück, bis zum Ende
- [ ] Die Unterhaltung führen, jedes Mal komplett mitschicken
- [ ] Abbrechen, Fehler, Wiederholen bei Rate-Limits
- [ ] Anhalten für eine Freigabe und später weitermachen
- [ ] Das alles weiter an den Browser streamen
- [ ] **… und das pro Anbieter**

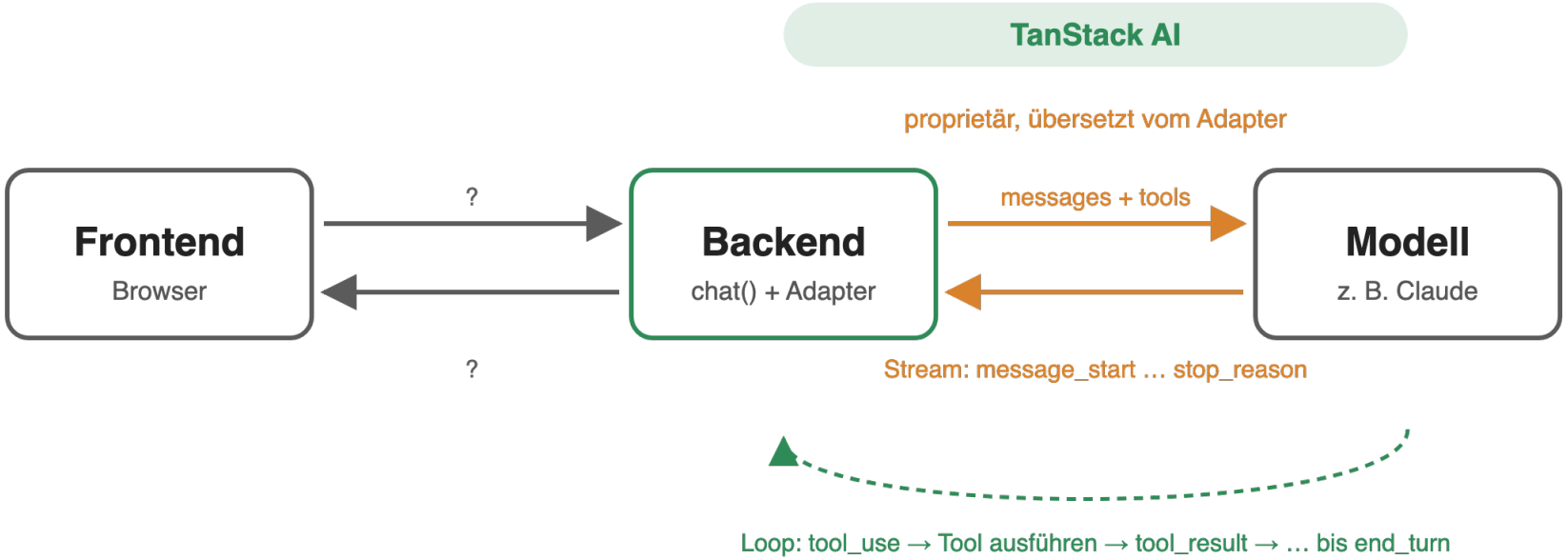
Wir sprechen mit dem Model im Backend…



TanStack AI übernimmt das Backend…

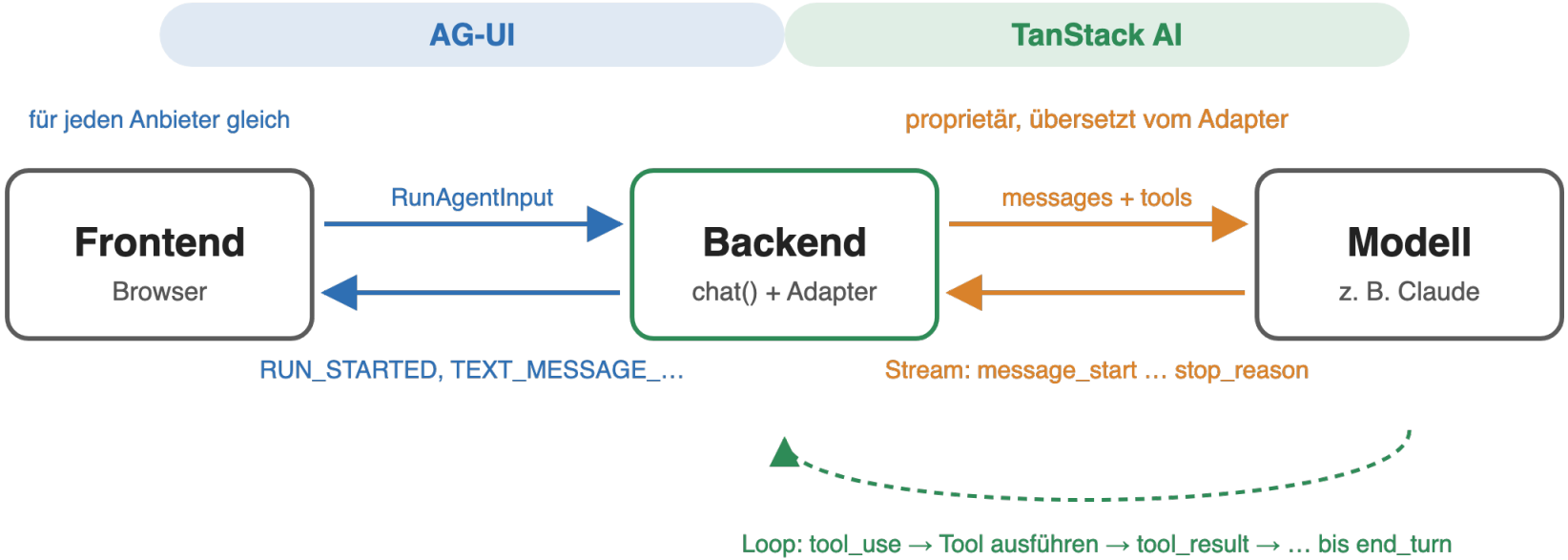


...und das Frontend mit dem Backend...

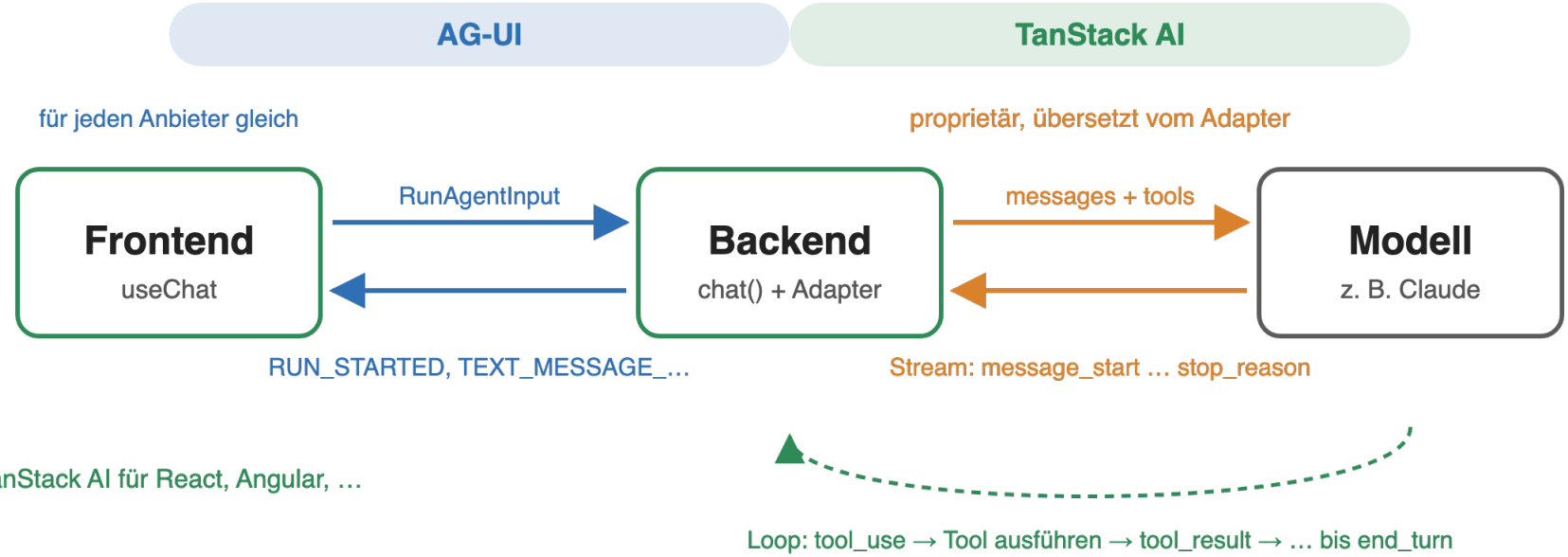


DIE STRECKE VOM FRONTEND ZUM MODEL

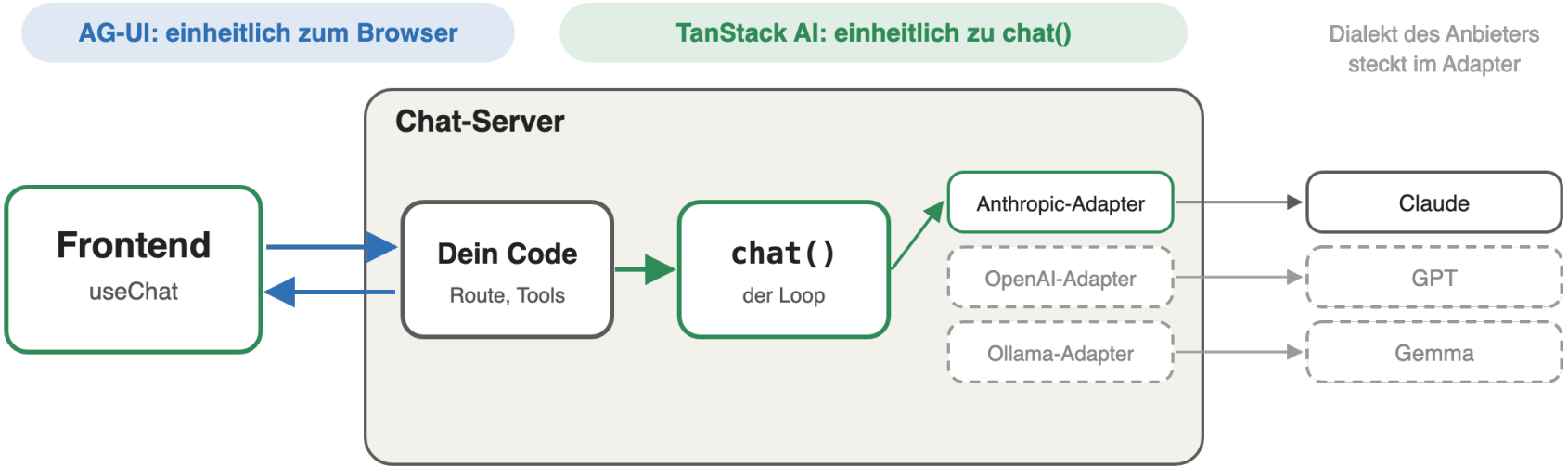
gesprochen über AG-UI



TanStack AI auch im Frontend



Der Dialekt steckt im Adapter



Modell wechseln heißt Adapter wechseln

Dein Code und das Frontend bleiben gleich (theoretisch).

Demo: Chat-Route mit chat()

- Die Route mit chat(), Adapter und threadId
- Aufruf mit curl: jetzt kommen AG-UI-Events
- Den Adapter wechseln: Claude oder Gemma über Ollama



Aus Events wird eine Nachricht

1. Der Chat-Server schickt AG-UI-Events: `RUN_STARTED`, `REASONING_…`, `TEXT_MESSAGE_CONTENT`, `TOOL_CALL_…`
2. Der Client setzt daraus eine `UIMessage` zusammen, eine Liste von Parts
3. Jedes Event ändert die Nachricht, danach wird neu gerendert, und die Antwort wächst
4. Die `message`-Komponente legt den Rahmen fest und rendert `<Parts />`
5. `<Parts />` wählt je Part die Komponente aus `partsComponents` nach `part.type`
6. Ein `tool-call`-Part geht nach dem Namen des Tools an `toolsComponents`

Beispiel: „Wie heißt du?“

AG-UI-Events vom Server

```
RUN_STARTED
REASONING_MESSAGE_START
REASONING_MESSAGE_CONTENT "Frage nach"
REASONING_MESSAGE_CONTENT " meinem Namen"
REASONING_MESSAGE_END
TEXT_MESSAGE_START
TEXT_MESSAGE_CONTENT "Ich bin"
TEXT_MESSAGE_CONTENT " der Assistent."
TEXT_MESSAGE_END
RUN_FINISHED
```

message in der Komponente

```
{
  role: "assistant",
  parts: [
    { type: "thinking",
      content: "Frage nach meinem Namen" },
    { type: "text",
      content: "Ich bin der Assistent." },
  ],
}
```

Aus vielen Deltas wird ein String je Part

Jeder Part findet seine Komponente

huetly-chat.tsx

```
createChatHook({
  components: {
    message: ({ message, Parts }) => {
      return <AssistantMessage>
        <Parts />
      </AssistantMessage>
    } },
  partsComponents: {
    thinking: ({ part }) => <details>…</details>,
    text: ({ part }) => <Markdown … />,
  },
})
```

Ablauf beim Rendern

```
message (role: assistant)
└ <Parts />
  ├ thinking → <details>
  └ text     → <Markdown>
```

`<Parts />` **sucht für jeden Part den Eintrag zu** `part.type`

Wer rendert was?

components.layout → `<div className="chat-messages">`

`<Messages />` ruft je Nachricht **components.message** auf

components.message
role: "user" → `<UserMessage>`

`<Parts />`

text
partsComponents
.text
→ `<Markdown>`

components.message
role: "assistant" → `<AssistantMessage>`

`<Parts />`

thinking
partsComponents
.thinking
→ `<details>`

text
partsComponents
.text
→ `<Markdown>`

`<Interrupts />` je wartender Freigabe **interruptsComponents.tools.antwortSenden**

gestrichelt: von TanStack AI · durchgezogen: deine Komponente

Demo: Antworten streamen

- Den Assistenten in die Anwendung einblenden
- useAppChat im Chat-Panel, Text-Parts als Markdown
- Im Netzwerk-Tab: die AG-UI-Events
- Neues Gespräch mit clear()



Eine Nachricht besteht aus Parts

```
UIMessage
{
  role: "assistant",
  parts: [
    { type: "thinking", content: "…"
},
    { type: "tool-call",
      name: "gesucheSuchen",
      state: "complete", output: […]
},
    { type: "text", content: "Ich
habe …" }
  ]
}
```

Der Client setzt Events zusammen

- Jedes TEXT_MESSAGE_CONTENT verlängert den text-Part
- Je Art von Part eine eigene Komponente
- Tool-Aufrufe und Denktext stehen in derselben Nachricht

Das Modell führt nichts aus, es fordert an

```
shared/tool-definitions.ts
export const gesucheSuchen = toolDefinition({
  name: "gesucheSuchen",
  description: "Sucht Gesuche. …",
  inputSchema: z.object({ tierart, ort,
zeitraum }),
  outputSchema: z.array(…),
});

// server/src/server-tools.ts
gesucheSuchen.server(async (filter) =>
  api.gesucheSuchen(filter));

// server/src/chat-server.ts
chat({ …, tools: [gesucheSuchen] });
```

- **Das Schema** ist der Vertrag mit dem Modell
- **.server()** ruft das bestehende Backend
- **chat()** führt das Tool aus und fragt das Modell erneut
- Die Pinnwand steckt im Toolnamen, nicht in einem Enum

Der Assistent liefert Daten, **die Komponente gehört der Anwendung**

Demo: Server-Tools

- Die Server-Tools gesucheSuchen und angeboteSuchen
- Das Tool-Ergebnis als Karte der Pinnwand rendern
- Die Devtools von TanStack AI: der Loop von chat()



Das Tool läuft im Browser

1 Lauf 1

Das Modell fordert gesuchVorbereiten an

2 Browser

Der Client führt das Tool aus und füllt das Formular

3 Lauf 2

Das Ergebnis geht zurück, das Modell antwortet

⚠️ **Fallstrick**

- chatOptions steht außerhalb jeder Komponente, deshalb router.navigate() statt useNavigate()

✅ **Keine Freigabe nötig**

- Das Formular hat seinen eigenen Knopf zum Absenden

Demo: Client-Tools

- Dieselbe Tool-Definition, aber mit .client()
- Das Formular für ein Gesuch öffnet sich vorbefüllt
- Im Netzwerk-Tab: zwei Requests



Erst fragen, dann handeln

Lauf 1

Das Modell fordert antwortSenden an

Pause

Der Lauf hält an, der Client meldet einen Interrupt

Mensch

Die Karte zeigt die Antwort, wie sie erscheinen wird

Lauf 2

resolveInterrupt setzt den Lauf fort

```
antwortSenden = toolDefinition({
  inputSchema: z.object({ eintragId,
text }),
  needsApproval: true,
});
```

- Ablehnen ist eine Antwort, auf die das Modell reagiert

Demo: Freigabe

- antwortSenden mit needsApproval
- Die Freigabekarte zeigt den Vorschlag
- Freigeben und Ablehnen
- Im Netzwerk-Tab: der zweite Request startet erst nach der Freigabe



Ein ehrlicher Ladezustand statt eines Spinners



Denktext

Der thinking-Part steht über der Antwort und füllt die Wartezeit



Laufende Tools

Jeder Tool-Aufruf hat einen Zustand, bis das Ergebnis da ist



Abbrechen

stop() beendet den Lauf mitten in der Antwort

Demo: Zwischenstände

- Den Denktext mit reasoning und thinking-Part anzeigen
- Eine Antwort mit stop() abbrechen
- Laufende Tools anzeigen



Wie weit ist TanStack AI?

1.0

seit dem 22. Oktober 2026

Der Kern ist stabil

createMCPServer

- Dieselben Tools als MCP-Server
- Fremde Agenten wie Claude Desktop oder Cursor rufen sie auf

Agent-Harness

- Ein Gespräch, das auf dem Server weiterlebt
- Ausdrücklich experimentell, Version 0.x

NILS HARTMANN
https://nilshartmann.net

Vielen Dank!

Slides & Code: https://react.schule/wjax-2026-tanstack-ai

Fragen und Kontakt:

nils@nilshartmann.net

https://nilshartmann.net/kontakt

